

Unit 06: Loop Statements

CONTENTS

Objectives

Introduction

6.1 Looping

6.2 for loop

6.3 While Loop

6.4 do-while loop

6.5 Jump and Break Statement

6.6 goto Statement

Summary

Keywords

Self-Assessment

Answer for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Explain looping concept in C
- Describe do-while loop
- Describe goto statement

Introduction

The C language includes a variety of program control statements that let you control the order of program execution. This unit discusses various iteration-based program control statements and how these can be implemented in a program. This unit also discusses some jump statements of C which are break and continue.

6.1 Looping

Iteration statements are also known as loops or looping statements because the program execution typically loops through the statement more than once. In this category, C provide the following statement, or you call loops.

1. for loop
2. while loop
3. do-while loop

Looping must not continue indefinitely as an analogy to real life you would not like to crack the same joke again and again, so a mechanism is required to break out the loop and to allow the executives of the next set of statements.

Therefore, a general structure has been devised for the implementation of a loop statement. Which can be more understood by understanding the various elements/ parts/components of a loop that controls the number of repetitions as given below:

1. Initial Expression(s): Initial expression(s) is usually an assignment expression(s) which initializes the control variable(s) of a loop, as they must be initialized before entering in a loop. The initial expression(s) is executed only once, in the beginning of the loop. But if this expression(s) occurs in the loop body, control variable(s) would be reassigned to initial values with every loop pass, and the condition expression would never fail.

2. Condition Expression: Conditional expression is typically a relational expression that is set up to terminate the execution of a loop. If the condition expression evaluates, to true i.e. 1, the loop body gets executed, otherwise the loop is terminated. A condition expression may be evaluated before entering in to a loop or before exiting from the loop called as entry-controlled loop and exit controlled loop respectively. In C, the for loop and while loop are entry-controlled loops whereas do while loop is exit-controlled loop.

3. Update Expression(s): The update expression(s) is essentially an increment expression or decrement expression that changes the value(s) of loop variable(s), so that they could come to the boundary values. The update expression(s) normally execute at the end of the loop body. It may appear in the body of loop as it is updating expressions that assign the variable a new updated value every time the loop passes.

4. The Loop Body: The loop body consists of statement(s) that is supposed to be executed again and again as long as the condition expression evaluator to true i.e. 1. In an entry-controlled loop, the condition expression evaluated first and if it evaluates to true, the loop-body is executed and if it evaluates to false, the loop-body is terminated. Whereas, in exit-controlled loop, the loop body executed first and then the condition expression are evaluated. It is evaluated to false i.e. 0, the loop is terminated, otherwise repeated.

The above-mentioned components are the essential component of a statement to be called as a loop statement. Missing any of them may change the basic meaning of a perfect loop. For, while and do-while statements of C, comprises of all these essential components, hence referred to as loop statements.

6.2 for loop

The for loop in C is the simplest, fixed and entry-controlled loop. It is simplest as the structure of for loop is divided into two segments i.e., control statement and the body of the loop. All its loop control elements are placed together in the control statement whereas body of the loop consists of statements to be executed repeatedly.

It is fixed as number of repetitions is known in advance and can be useful in a situation when you want to do something a fixed number of times. It is an entry-controlled loop as the control statement placed before the loop body i.e. condition expression will be evaluated first. The general form of the for loop is:

`for(initial expression(s) ; condition expression ; update expression(s)) loop-body;`



Consider the following statement:

```
for ( i =1 ; i<= 10; ++i)
```

```
printf(" \n Hello World!");
```

where `i` is an integer variable declared already.

`i=1;` is an initial expression.

`i<= 10;` is a conditional expression.

`++i;` is an update expression.

And the statement

```
printf("\n Hello World!");
```

is the body of the loop

When the above statement is encountered during program execution, the following events occur:

1. Initial expression is evaluated first and i will be assigned an initial value 1 i.e. $i=1$.
2. Then the condition expression is evaluated i.e. $i \leq 10$ and the result will be true as $1 \leq 10$ is true.
3. Since the condition expression is true, the statement in the loop body is executed i.e. `printf("\n Hello World!");` which prints the message Hello World! on the screen.
4. After the execution of the loop body, the update expression i.e. $++i$ is executed which increment the value of i by 1. In this way after the first execution of the loop the value of i becomes 2 as initially it was 1.
5. After the execution of the update expression the condition expression is again evaluated. If it evaluates to true the sequence is repeated from step no. 3, otherwise the loop terminates.



After a certain repetition the condition expression evaluates to false, as the value of i will be greater than 10, then loop will be terminated and the output of the code

may appear as:

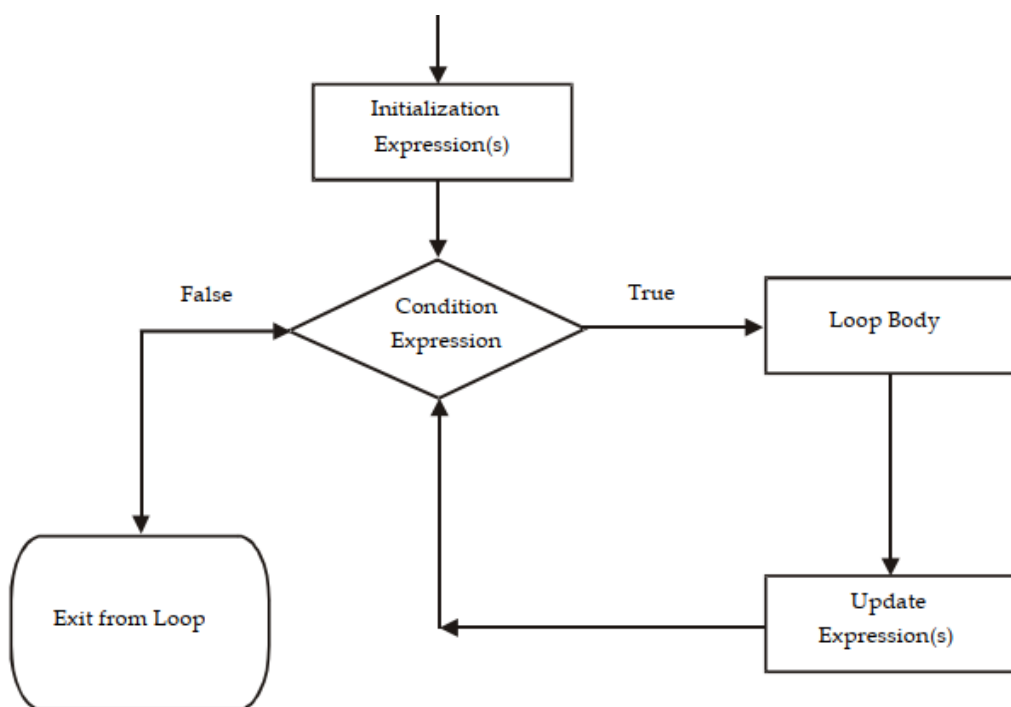
Hello World!

Hello World!

::

Hello World! (10 Times)

Also note that the loop body never executes if condition expression is evaluated to false in its first execution. Figure shows the operation of a for loop.



Program :-

```
#include<stdio.h>
```

```

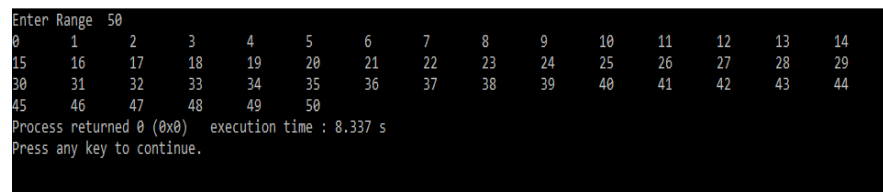
int main(){

    inti,range;
    printf("Enter Range");
    scanf("%d",&range);
    for(i=0;i<=range;i++)
    {

        // printf("%d\n",i);
        printf("%d\t",i);
    }
    return 0;
}

```

Output : -



```

Enter Range 50
0   1   2   3   4   5   6   7   8   9   10  11  12  13  14
15  16  17  18  19  20  21  22  23  24  25  26  27  28  29
30  31  32  33  34  35  36  37  38  39  40  41  42  43  44
45  46  47  48  49  50
Process returned 0 (0x0) execution time : 8.337 s
Press any key to continue.

```



Program :

```

#include<stdio.h>

int main(){

    inti,n;
    printf("Enter Number");
    scanf("%d",&n);
    for(i=1;i<=10;i++)
    {
        printf("%d*%d=%d\n",i,n,i*n);

    }
    return 0;

}

```

Output : -

```

Enter Number 5
1*5=5
2*5=10
3*5=15
4*5=20
5*5=25
6*5=30
7*5=35
8*5=40
9*5=45
10*5=50

Process returned 0 (0x0)   execution time : 3.716 s
Press any key to continue.

```

6.3 While Loop

The second type of loop, the while loop is an entry-controlled loop as it tests the conditions first and if the condition is true, then only the control will enter into the loop body.

When each iteration of the loop is finished, the control returns to the while statement which perform the condition test again as so on. But if the condition is false the first time, no iteration of the loop executes, and control passes to the statement next to loop statement. In this way, it is a sort of variable loop as we do not know the exact number of iterations. The statements repeat over and over until certain specified conditions are met.

The while loop has the following form:

```
while (condition expression)
```

```
loop body;
```

Where the loop-body may contain a single statement, a compound statement or an empty statement. The while loop iterates the loop body as long as the specified condition expression evaluates to true.

The while loop doesn't explicitly contain the initialization expression and update expressions of the loop. These two expressions are normally provided by the programmers as the initialization expression(s) should be placed before the loop begins and updating expression(s) should be inside the loop body. By using all these expressions, the general form of while loop may look like as:

```

:
initialization expression(s);
while (conditional expression)
{
: Loop Body
upadation expression;
}

```



Consider the following segment of code:

```

i = 1 ;
while ( i <= 10)
{

```

```
printf("\n Hello World!");
+ + i;
}
```

where i is an integer variable declared already

i = 1; is an initial expression

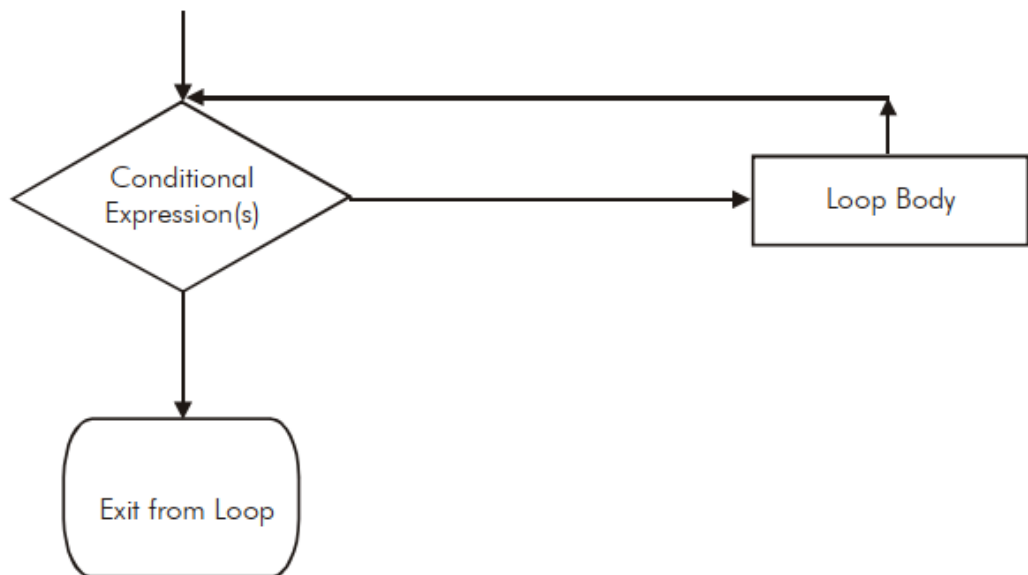
i <= 10; is a conditional expression

++i ; is an update expression.

And the statements between the { and } forms the body of the loop. But the braces can be discarded, if there is only one statement in the loop body

When the program execution reaches a while statement, the following events occur:

1. First of all the conditional expression is evaluated i.e., $i < 10$.
2. The conditional expression is evaluated to true as i was 1 initially and $1 < 10$ is true. But if it evaluates to false, the loop will be terminated, and the control moves to the first statement following loop body.
3. Since the condition expression is true, the loop body will be executed i.e., the print statement and the updating expression.
4. With the closing braces (}), it is assumed that the loop is finished and the control moves back to the while statement, which repeats the test again and proceeds accordingly.



Program : -

```
#include<stdio.h>

int main(){
  int number;
  printf("Enter Number");
  scanf("%d",&number);
  while(number<=50)
  {
```

```
printf("Value of number =%d\n",number);
    number++;
}
return 0;
}
```

Output : -

```
Enter Number 20
Value of number =20
Value of number =21
Value of number =22
Value of number =23
Value of number =24
Value of number =25
Value of number =26
Value of number =27
Value of number =28
Value of number =29
Value of number =30
Value of number =31
Value of number =32
Value of number =33
Value of number =34
Value of number =35
Value of number =36
Value of number =37
Value of number =38
Value of number =39
Value of number =40
Value of number =41
Value of number =42
Value of number =43
Value of number =44
Value of number =45
Value of number =46
Value of number =47
Value of number =48
```



Program : -

```
#include<stdio.h>

int main()
{
    int n, reverse=0, rem;
    printf("Enter a number: ");
    scanf("%d", &n);
    while(n!=0)
    {
        rem=n%10;
        reverse=reverse*10+rem;
        n/=10;
    }
    printf("Reversed Number: %d",reverse);
    return 0;
}
```

Output : -

```
Enter a number: 1024
Reversed Number: 4201
Process returned 0 (0x0)   execution time : 3.135 s
Press any key to continue.
```

6.4 do-while loop

C's third loop statement is the do-while loop, is an exit controlled loop i.e. it tests the conditions after having executed the statement with in loop body. This means unlike the for and while loops, a do while loop always executes at least once. The statement of the do-while loop is as follows:

```
do
{
loop-boody ;
}while (conditional expression) ;
```

The braces { } can be discarded when the loop-boody contains a single statement. The do-while loop iterates the loop body as long as the specified condition is true while testing the condition at the end of the loop each time, rather than at the beginning, as is done by the for and the while loop.

Like while loop, do-while loop also doesn't contain the initialization and Updation expression as part of loop statement. However, these expressions can be associated with do-while loop by the programmer according to required logic. Then the new form of do-while loop may looks like as:

```
Initialization expression(s);
do
{
Loop body;
Updating expression;
}while ( conditional expression(s) );
```



Consider the following segment of code :

```
i=1;
do
{
printf("\n Hello World!");
++i;
} while ( i<=10);
where i is an integer variable declared already
i=1; is an initial expression.
i<= 10; is a conditional expression.
++i; is an update expression.
```

When the program control reaches at a 'do while' loop, the following events occur:

1. The loop body will be executed i.e. the print statement and the updation statement.
2. The conditional expression will be evaluated i.e. $i \leq 10$.
3. The conditional expression will evaluates to true as the value as i is 2 this time (initially=1).
4. Since the condition expression is true, the control will move back to execute the loop-body once again.



After a certain repetition the condition expression will evaluate to false as the value as I will be greater than 10, the loop will be terminated.

The output of the above code may look like as:

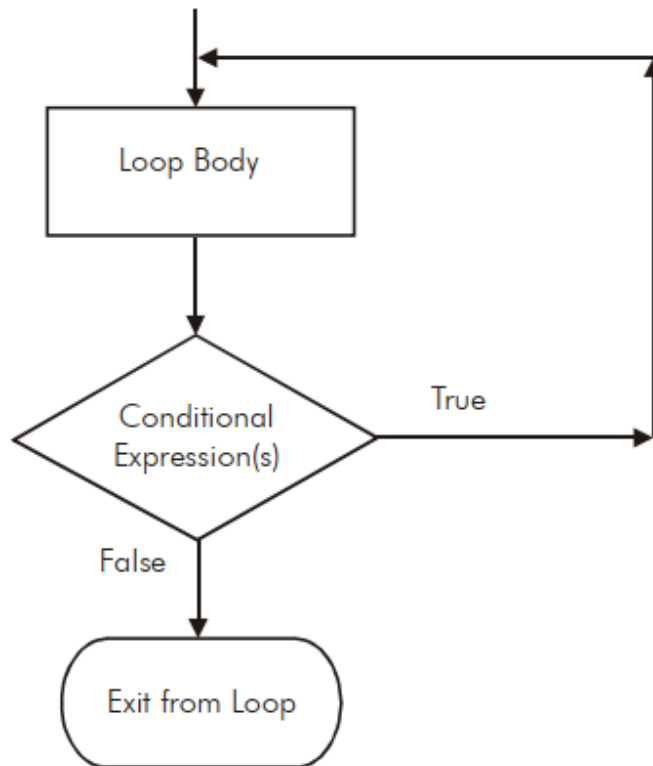
Hello World !

Hello World !

:

Hello World ! (10 times)

Figure demonstrate working of do-while loop



The do-while loop is well suited for the problems where number of repetitions is not known in advance. But this is sure that the loop will be executed at least once. The following segment of codes will give a clear picture of the concept:

```

for ( i = 11; i<=10; i++)
printf("\n%d",i);
printf ("\n Thank you");

```

Output:

Thank you

i=11;

while (i<=10)

{

printf("\n%d", i);

++i;

```
}  
printf("\n Thank you");
```

Output:

Thank you

```
i=11;  
do  
{  
printf("\n % d", i);  
++i;  
} while (i<=10);  
printf("\n Thank you");
```

Output:

11

Thank you

It's obvious from the code above that a do-while loop is always executed at least once, regardless of the condition's outcome. Since the condition expression is evaluated at the end of the loop rather than at the beginning.

Program to print series of number after enter first number



Program :-

```
#include<stdio.h>  
  
int main(){  
  
    int number;  
    printf("Enter a number:");  
    scanf("%d",&number);  
    do{  
  
        printf("Value of number is= %d\n",number);  
        ++number;  
  
    }while(number<=20);  
    return 0;  
  
}
```

Output:-

```

Enter a number:10
Value of number is= 10
Value of number is= 11
Value of number is= 12
Value of number is= 13
Value of number is= 14
Value of number is= 15
Value of number is= 16
Value of number is= 17
Value of number is= 18
Value of number is= 19
Value of number is= 20

Process returned 0 (0x0)   execution time : 21.635 s
Press any key to continue.

```

Let's write another interesting program using do-while loop. This program will print the alphabet A to Z along with their ASCII codes.



Program : -

```

#include<stdio.h>

void main()
{
    int a,b;
    char choice ;
    do
    {
        printf("\n Main Menu ");
        printf("\n -----");
        printf("\n1. ADDITION ");
        printf("\n2. SUBTRACTION ");
        printf("\n3. QUIT ");
        printf("\n Enter your choice:");
        scanf("%c",&choice);
        if( choice == '1' || choice == '2')
        {
            printf("\n Enter the first & second number :");
            scanf("%d %d", &a,&b);
        }
        switch (choice)
        {
            case '1': printf("\n Result = % d", a + b);
            break;

```

```

case '2' : printf("\n Result = % d", a - b);
break;
default : printf("\n Wrong input");
}
} while (choice != '3' );
printf("\n Thank you");
}

```

Output:

Run 1:

Main Menu

1. ADDITION

2. SUBTRACTION

3. QUIT

Enter your choice: 1↵

Enter the first and second number : 5 6↵

Result = 11

Main Menu

1. ADDITION

2. SUBTRACTION

3. QUIT

Enter your choice : 9↵

wrong input

Main Menu

1. ADDITION

2. SUBTRACTION

3. QUIT

enter your choice : 3↵

Thank you



To come out from the program execution if the choice entered is other than the given one, the menu is simply redisplayed until user entered the correct one. For the desired operation, program required the input accordingly

6.5 Jump and Break Statement

What if you need to exit from a loop statement even before the test condition becomes false? You can use the break statement. The break statement is used to terminate loops or to exit from a switch (discussed later). When break is encountered inside any C loop, the loop is immediately exited without testing the loop condition and control automatically passes to the first statement after the loop. It can be used within a while, a do-while, a for or a switch statement. The break statement is written simply as

```
break;
```

The break statement does not have any operand.

Following C code snippets illustrate use of break statement to exit from various C loops. In each situation, the loop will continue to execute as long as the current value for the integer variable x does not exceed 10. However, the computation will break out of the loop if a negative value for x is detected.

While loop

```
scanf ("%d", &x);
while (x <= 10)
{
    if (x < 0)
    {
        printf ("Negative value entered!!\n");
        break;
    }
    scanf ("%d", &x);
}
```

do-while loop

```
do
{
    scanf ("%d", &x);
    if (x < 0)
    {
        printf ("Negative value entered");
        break;
    }
} while (x <= 10);
```

for loop

```
for (i = 1; x <= 10; ++i)
{
    scanf ("%f", &x);
    if (x < 0)
    {
        printf ("Negative value entered!!!");
    }
}
```

```
break;
}
}
```

When break is used in nested while, do-while, for or switch statements, it will cause a transfer of control out of the immediate enclosing statement, but not out of the outer surrounding statements.

Consider the following code snippet in which a while loop is nested within a for loop.

```
for (i = 0; i <= n; ++i)
{
    while ((c = getchar()) != '\n')
    {
        if (c == '*') break;
    }
}
```

The internal while loop terminates if the character variable c is assigned an asterisk (*). However, the for loop will continue to execute. Thus, if the value of i is less than n when the break occurs, the program will increment i and make another pass through the for loop.



Write a program to print all prime numbers from 1 to 300. (Use nested loops, break and continue)

6.6 goto Statement

In earlier programming languages goto was very popular looping construct to branch to one particular statement from another one unconditionally. No condition is checked for looping directly. Due to inherent problems associated with goto branching its use is generally discouraged.

For the reasons of backward compatibility, C supports the goto statement to branch unconditionally from one point to another in the program. A goto statement uses an identifier, called label, which specifies the statement to which branching would start execution after a goto has been encountered. A label is any valid identifier name, and must be followed by a colon. A label is placed immediately before the statement where the control is to be transferred.

The general forms of goto and label statements are shown below:

```
                goto label;

.....

.....

label: statements;

                .....
                statement;
```

The label can be anywhere in the program either before or after the goto label; statement.



There is no built-in mechanism for the flow of execution to come back from where it branched.



```
/*Program to demonstrate working of goto statement*/
```

```
#include<stdio.h>
```

```
int main(){
```

```
    inti;
```

```
    for(i=0;i<=10;i++)
```

```
    {
```

```
        if(i==5)
```

```
        {
```

```
            goto STATUS;
```

```
        }
```

```
        printf("%d\n",i);
```

```
    }
```

```
    STATUS:
```

```
    printf("Value of i is = 5 than goto working");
```

```
    return 0;
```

```
}
```

Output :-

```
0
1
2
3
4
Value of i is = 5 than goto working
Process returned 0 (0x0) execution time : 0.013 s
Press any key to continue.
```

Summary

- The for loop in C is the simplest, fixed and entry-controlled loop. An infinite for loop can be created by skipping the conditional expression.
- A conditional expression cannot have multiple expression like initialization and Updating expression, but it may contain several conditions linked together using logical operators.
- The second type of loop, the while loop is an entry-controlled loop as it tests the conditions first and if the condition is true, then only the control will enter into the loop body. An empty loop can also be configured using while statement and could used as a time delay loop.

- C's third loop statement is the do while loop, is an exit controlled loop i.e. it tests the conditions after having executed the statement with in loop body. Unlike the for and while loops, a do while loop always executes at least once.
- The break statement is used in a program to skip the particular part of program code. The jump statement continue is the compliment of the break statement.
- Instead of forcing termination, it causes the control to jump to the beginning of the loop. The goto statement is C's another jump statement which causes a program control to jump immediately to an executed statement elsewhere in the function.

Keywords

Control Statements: The statements that allow programmers to alter the sequential flow of execution of the program and control the flow are called control statements.

For Loop: A for loop allows execution of a statement (or a block of statements) repeatedly a number of times.

While Loop: In case the number of times a statement is to be executed is not known in advance, while loop is used.

goto statement: The goto statement is known as jump statement in C.

Self-Assessment

1. Choose a right C Statement.
 - A. Loops or Repetition block executes a group of statements repeatedly.
 - B. Loop is usually executed as long as a condition is met.
 - C. Loops usually take advantage of Loop Counter
 - D. All the above.
2. Choose a correct C for loop syntax.
 - A. for(initialization; condition; increment operation)


```
{
    //statements
}
```
 - B. for(declaration; condition; increment operation)


```
{
    //statements
}
```
 - C. for(declaration; increment operation; condition)


```
{
    //statements
}
```
 - D. for(initialization; condition; increment operation;)


```
{
    //statements
}
```


3. How many times Hello be printed in the following program?

```
#include <stdio.h>
int main()
{
    inti;
    for (i=0; i<=10; i++)
        printf("Hello");
    return 0;
}
```

- A. 100
- B. 10
- C. 11
- D. Infinite

4. What will be the output of the following C code?

```
#include <stdio.h>
void main()
{
    int k = 0;
    for (k)
        Printf ("Hello");
}
```

- A. Compile time error
- B. Hello
- C. Nothing
- D. Varies

5. What is the output of this program?

```
void main()
{
    int x=0;
    for(;;)
    {
        if(x==3)
            break;
        printf("%d ",++x);
    }
}
```

- A. 1 2 3

- B. 0 1 2 3
- C. 1 2
- D. Compilation Error
6. Which loop is guaranteed to execute at least one time.
- A. for
- B. while
- C. do while
- D. None of the above
7. do-while loop terminates when conditional expression returns?
- A. One
- B. Zero
- C. Non - zero
- D. None of the above
8. Which of the following statement about for loop is true?
- A. Index value is retained outside the loop
- B. Index value can be changed from within the loop
- C. Goto can be used to jump, out of the loop
- D. All of these
9. Which one is correct syntax for do-while loop?
- A. do
- ```
{
 //Statements

}while(condition test);
```
- B. do,
- ```
{  
    //Statements  
  
}while(condition test);
```
- C. do,
- ```
{
 //Statements

}while(do);
```
- D. do
- ```
{
```

//Statements

}while(do);

10. How many times Programming will be printed?

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
inti = -5;
```

```
while (i<= 5)
```

```
{
```

```
if (i>= 0)
```

```
break;
```

```
else
```

```
{
```

```
i += 1;
```

```
continue;
```

```
}
```

```
printf("Programming");
```

```
}
```

```
return 0;
```

```
}
```

- A. 5 times
- B. 10 times
- C. 0 times
- D. Infinite times

11. The break statement is used to exit from:

- A. a DO loop.
- B. a FOR loop.
- C. a SWITCH statement.
- D. all of above.

12. The GOTO statement is used:

- A. to permit two different expressions to appear in situation where only one expression would ordinarily used.
- B. to terminate loops or to exit from a switch.
- C. an unconditional transfer of control to a named label.
- D. to carry out a logical test and then take one of two possible actions, depending upon the outcome of a test.

13. Destination is specified by a label within the statement, is called

- A. While loop

Programming In C

- B. Goto statement
 - C. For loop
 - D. For each loop
14. Which from the following is a jumping statement?
- A. Break statement
 - B. Continue statement
 - C. Switch statement
 - D. All of them
15. Which keyword is used to come out of a loop only for that iteration?
- A. break
 - B. continue
 - C. return
 - D. none of the mentioned

Answer for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. D | 2. A | 3. C | 4. A | 5. A |
| 6. C | 7. B | 8. D | 9. A | 10. C |
| 11. D | 12. C | 13. B | 14. D | 15. B |

Review Questions

1. What do you mean by looping?
2. Describe for loop with the help of suitable example.
3. Differentiate while loop and do-while loop.
4. What is the advantage of break statement in while loop?
5. Write a program to find the factorial value of any number entered through the keyboard.
6. Write a program to print all the ASCII values and their equivalent characters using a while loop. The ASCII values vary from 0 to 255.
7. Write a program to find the range of a set of numbers. Range is the difference between the smallest and biggest number in the list.
8. Write a program to calculate overtime pay of 10 employees. Overtime is paid at the rate of ₹ 12.00 per hour for every hour worked above 40 hours. Assume that employees do not work for fractional part of an hour.
9. Write a program to print out all Armstrong numbers between 1 and 500. If sum of cubes of each digit of the number is equal to the number itself, then the number is called an Armstrong number. For example, $153 = (1 * 1 * 1) + (5 * 5 * 5) + (3 * 3 * 3)$
10. Write a program to enter the numbers till the user wants and at the end it should display the count of positive, negative and zeros entered.
11. Write a program to find the range of a set of numbers. Range is the difference between the smallest and biggest number in the list.



Further Readings

Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008.

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi.

Byron Gottfried, "Programming with C", Tata McGraw Hill Publishing Company Limited, New Delhi.

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

YashvantKanetkar, Let us C



Web Links

<https://www.tutorialspoint.com/index.htm>

www.webopedia.com

www.web-source.net

Unit 07: Functions

CONTENTS

Objectives

Introduction

7.1 Need for User-defined Function

7.2 A Multifunction Program

7.3 Elements of User-defined Functions

7.4 Definition of Functions

7.5 Return Value and their Types

7.6 Function Calls

7.7 Function Declaration

7.8 Category of Functions

7.9 Argument but no Return Values

7.10 Arguments with Return Values

7.11 Functions that Return Multiple Values

7.12 Function Prototype

7.13 Recursive Functions

7.14 Library functions

Summary

Keywords

Self-Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- State the need for user defined functions
- Identify category of functions
- Describe functions that return multiple values
- Discuss recursive functions

Introduction

A function is a programming unit with a unique name that may be identified. It can be invoked by a programme once it has been defined. When called, it may accept zero or more inputs. The code placed inside the function specification determines what should be done with the incoming input(s). The function generates a single output after doing the given transformation. The caller of the function receives this output.

7.1 Need for User-defined Function

Why write separate functions at all? Why not squeeze the entire logic into one function, main()?

Two reasons:

1. Writing functions avoids rewriting the same code over and over. Suppose you have a section of code in your program that calculates area of a triangle. If later in the program you want to calculate the area of a different triangle, you won't like it if you are required to write the same instructions all over again. Instead, you would prefer to jump to a 'section of code' that calculates area and then jump back to the place from where you left off. This section of code is nothing but a function.
2. Using functions it becomes easier to write programs and keep track of what they are doing. If the operation of a program can be divided into separate activities, and each activity placed in a different function, then each could be written and checked more or less independently. Separating the code into modular functions also makes the program easier to design and understand.

7.2 A Multifunction Program

The use of a function is one of the advantages of the C programming language. Always behave like a standard function or method in C. One function may call another, and so on. In C, there are no limitations on the number of functions that can be called in a programme. It is preferable to decompose the complex problem into tiny, easily manageable parts and develop a function. The control will be passed from the calling programme part to the called function block. If the called function is successfully executed, control will be returned to the programme segment that called it. There is always overhead of a transfer of the control between calling portion and a called function block.



Example: The multifunction program segment is shown below

```
function1 ()
{
    -----
    -----

function2 ();
    -----
    -----

function4 ();
}
function2 ()
{
    -----
    -----

function3 ();
    -----
    -----

}
function3 ()
{
    -----
    -----

}
function4 ()
{
    -----
```

 }



: A program to demonstrate the transfer of control between the multifunction

program.

Main()

```
{
int j = 10;
printf ("Inside the main() function\n");
function1 ();
printf ("after the function 1\n");
printf ("main function () \n");
printf ("j = %d\n", j);
}
```

function1 ()

```
{
inti,n;
n = 3;
for(i = 0; i<=n-1; ++){
printf("inside a function 1\n");
printf("i = %d\n", i);
function2 ();
}
}
```

function2 ()

```
{
printf ("transfer of control\n");
printf ("inside a function 2\n");
}
```

7.3 Elements of User-defined Functions

Functions are classified as one of the derived data types in C. We can therefore define functions and use them like any other variables in C programs. It is therefore not a surprise to note that there exist some similarities between functions and variables in C.

1. Both function names and variable names are considered identifiers and therefore they must adhere to the rules for identifiers.
2. Like variables, functions have types associated with them.
3. Like variables, function names and there must be declared and defined before they are used in a program.

In order to make use of a user-defined function, we need to establish three elements that are related to functions.

1. Function definition.
2. Function call

3. Function declaration.

The function definition is an independent program, module that is specially written to implement the requirements of the function. In order to use this function we need to invoke it in a required place in the program. This is known as the function call. The program that calls the function is referred to as calling program or calling function.

7.4 Definition of Functions

A function is a standalone piece of executable code that can be called from any other function. The idea of functions comes to mind in many systems when a group of statements must be executed repeatedly at various points in the programme and possibly with different sets of data. Those repeating statements are stored in a function and called as needed. When a function is called, control is sent to the called function, which is then run, before being returned to the calling function (to the statement following the function call). Let us see an example as shown below:

Example:

```
/* Program to illustrate a function */
#include <stdio.h>
main ()
{
    void sample ();
    printf("\n You are in main");
}
void sample ()
{
    printf("\n You are in sample");
}
```

Output:

You are in sample

You are in main

Here we are calling a function sample () through main () i.e. control of execution transfers from main () to sample () , which means main () is suspended for some time and sample () is executed. After its execution the control returns back to main () , at the statement following function call and the execution of main () is resumed.

The syntax of a function is:

```
return data type function_name (list of arguments)
{
    datatype declaration of the arguments;
    executable statements;
    return (expression);
}
```

where,

1. Return data type is the same as the data type of the variable that is returned by the function using return statement.
2. A function_name is formed in the same way as variable names/identifiers are formed.
3. The list of arguments or parameters are valid variable names as shown below, separated by comma
4. Arguments give the values which are passed from the calling function.

5. The body of function contains executable statements.
6. The return statement returns a single value to the calling function. ommas: (data type1 var1,data type2 var2,..... data type n var n) for example (int x, float y, char z).



Define the functions factorial(), prime() and Fibonacci() in a file, say 'myfuncs.c'. Do not define main() in this file.



Example: Let us write a simple function that calculates the square of an integer.

```
/*Program to calculate the square of a given integer*/
/* square() function */
{
int square (int no) /*passing of argument */
int result ; /* local variable to function square */
result = no*no;
return (result); /* returns an integer value */
}
/*It will be called from main()as follows */
main()
{
int n ,sq; /* local variable to function main */
printf ("Enter a number to calculate square value");
scanf ("%d",&n);
sq=square(n); /* function call with parameter passing */
printf ("\nSquare of the number is : %d", sq);
} /* program ends */
```

Output:

Enter a number to calculate square value: 5

Square of the number is: 25

7.5 Return Value and their Types

If a function has to return a value to the calling function, it is done through the return statement.

It may be possible that a function does not return any value; only the control is transferred to the calling function. The syntax for the return statement is:

```
return (expression);
```

We have seen in the square() function, the return statement, which returns an integer value.

Important Points

1. You can pass any number of arguments to a function but can return only one value at a time.



Example: The following are the valid return statements

- (a) return (5);
- (b) return (x*y);



Example: The following are the invalid return statements

- (a) `return (2, 3);`
- (b) `return (x, y);`

2. If a function does not return anything, void specifier is used in the function declaration.

Example:

```
void square (int no)
{
    intsq;
    sq = no*no;
    printf ("square is %d", sq);
}
```

3. All the function's return type is by default is "int", i.e. a function returns an integer value, if no type specifier is used in the function declaration.

Examples:

- (a) `square (int no); /* will return an integer value */`
- (b) `int square (int no); /* will return an integer value */`
- (c) `void square (int no); /* will not return anything */`

4. What happens if a function has to return some value other than integer? The answer is very simple: use the particular type specifier in the function declaration.

Example: Consider the code fragments of function definitions below:

(a) Code Fragment – 1:

```
charfunc_char( ..... )
{
    char c;
    .....
    .....
    .....
}
```

(b) Code Fragment – 1:

```
floatfunc_float (.....)
{ fl
    oat f;
    .....
    .....
    .....
    return(f);
}
```

Thus from the above examples, we see that you can return all the data types from a function, the only condition being that the value returned using return statement and the type specifier used in function declaration should match.

5. A function can have many return statements. This thing happens when some condition

based returns are required.

Example:

```
/*Function to find greater of two numbers*/
int greater (int x, int y)
{
    if (x>y)
        return (x);
    else
        return (y);
}
```

6. And finally, with the execution of return statement, the control is transferred to the calling function with the value associated with it.

In the above example, if we take $x = 5$ and $y = 3$, then the control will be transferred to the calling function when the first return statement will be encountered, as the condition $(x > y)$ will be satisfied. All the remaining executable statements in the function will not be executed after this returning.

7.6 Function Calls

A function can be called by supplying its name followed by a list of arguments separated by commas and surrounded in parentheses. If a function call does not require any parameters, it must be followed by an empty pair of parentheses.

The arguments appearing in the function call are referred to as actual arguments, in contrast to the formal arguments that appear in the first line of function definition.



e.g.: /* Program to find square of given number */

```
main()
{
    float square (float); /* function prototype dec/n*/
    float a, b;
    printf ("\n Enter the number:");
    scanf ("%f", &a);
    b = square (a); /* calling of function with */
    /* actual arguments */
    printf ("Square of entered no. is = %f" , b);
}

float square (x) /* function definition with format l argument */
float x; /* format l argument declaration */
{
    float y; /* Local variable declaration
    y = x * x;
    return (y);
}
```

Output:

Enter the number: 2

Square of the entered number is = 4

7.7 Function Declaration

A function is declared in the following manner:

```
<return_data_type><function_name>(arg1, arg2, arg3)
<data_type_1> arg1; <data_type_2> arg2; <data_type_3> arg3;
{
statement-1;
statement-2
:
:
statement-n;
return(<expression of return_data_type>);
}
```

Example: The following function (name being getsq) returns the square of the input number of float type. Clearly the <return_data_type> will also be float type.

```
floatgetsq(x)
float x;
{
return(x*x);
}
```

Another form of a function definition is:

```
<return_data_type><function_name>(formal argument list)
{
statement-1;
statement-2;
-----
statement-n;
return (<expression of return_data_type>);
}
```

Where formal argument list is a comma separated list of variables and their corresponding data types.

The following function, addthem, takes two int type arguments and returns the sum of the two.

```
intaddthem(int a, int b)
{
return(a+b);
}
```

The <return_data_type> always represents the data type of the value which is returned. The type specification can be omitted if the function returns an integer or a character.

An empty pair of parenthesis must follow the function name if the function definition does not include any arguments.

The argument declarations follow the first line. Each formal argument must have the same data type as its corresponding actual argument.

The remainder of the function definition is a compound statement that defines the action to be taken by the function. It is referred to as the body of the function.

The last statement in the body of function is return (expression). It is used to return the computed result, if any, to the calling program.



What would be the output of this program?

```
Main( )
{
    printf("\nOnly stupids use C?");
    Display( );
}
display( )
{
    Printf("\nFools too use C!");
    Main( );
}
```

7.8 Category of Functions

We categorize a function's invoking (calling) depending on arguments or parameters and their returning a value. In simple words, we can divide a function's invoking into four types

depending on whether parameters are passed to a function or not and whether a function returns some value or not.

The various types of calling functions are:

1. With no arguments and with no return value.
2. With no arguments and with return value.
3. With arguments and with no return value.
4. With arguments and with return value.

11.9 No Argument and no Return Values

Any function which has no arguments and does not return any values to the calling function, falls in this category. These type of functions are confined to themselves i.e. neither do they receive any data from the calling function nor do they transfer any data to the calling function.

So there is no data communication between the calling and the called function are only program control will be transferred.



/* Program for illustration of the function with no arguments and no return

value*/

/* Function with no arguments and no return value*/

#include <stdio.h>

main()

{

void message();

printf("Control is in main\n");

Programming Methodologies

```

message(); /* Type 1 Function */
printf("Control is again in main\n");
}
void message()
{
printf("Control is in message function\n");
} /* does not return anything */

```

Output:

Control is in main

Control is in message function

Control is again in main

7.9 Argument but no Return Values

If a function includes arguments but does not return anything, it falls in this category. One way communication takes place between the calling and the called function.

Before proceeding further, first we discuss the type of arguments or parameters here. There are two types of arguments:

1. Actual arguments
2. Formal arguments

Let us take an example to make this concept clear:

Example: Write a program to calculate sum of any three given numbers.

```

#include <stdio.h>
main()
{
int a1, a2, a3;
void sum(int, int, int);
printf("Enter three numbers: ");
scanf ("%d%d%d",&a1,&a2,&a3);
sum (a1,a2,a3); /* Type 3 function */
}
/* function to calculate sum of three numbers */
void sum (int f1, int f2, int f3)
{
int s;
s = f1+ f2+ f3;
printf ("\nThe sum of the three numbers is %d\n",s);
}

```

Output

Enter three numbers: 23 34 45

The sum of the three numbers is 102

Here f1, f2, f3 are formal arguments and a1, a2, a3 are actual arguments.

Thus we see in the function declaration, the arguments are formal arguments, but when values are passed to the function during function call, they are actual arguments.

7.10 Arguments with Return Values

In this category, two-way communication takes place between the calling and called function i.e. a function returns a value and also arguments are passed to it. We modify above example according to this category.



Example: Write a program to calculate sum of three numbers.

```
/*Program to calculate the sum of three numbers*/
#include <stdio.h>

main ()
{
    int a1, a2, a3, result;
    int sum(int, int, int);
    printf("Please enter any 3 numbers:\n");
    scanf ("%d %d %d", &a1, &a2, &a3);
    result = sum (a1,a2,a3); /* function call */
    printf ("Sum of the given numbers is : %d\n", result);
}

/* Function to calculate the sum of three numbers */
int sum (int f1, int f2, int f3)
{
    return(f1+ f2 + f3); /* function returns a value */
}
```

Output

Please enter any 3 numbers:

3 4 5

Sum of the given numbers is: 12

7.11 Functions that Return Multiple Values

The compiler will raise a compilation error if a function is not called with the required number of properly typed arguments. When calling a function, there are two methods for passing arguments to it: call by value and call by reference.

Call by Value

Call by value means sending the values of the arguments to functions. When a single value is passed to a function via an actual argument, the value of the actual argument is copied into the function. Therefore, the value of the corresponding formal argument can be altered within the function, but the value of the actual argument within the calling routine will not change. This procedure for passing the value of an argument to a function is known as passing by value or call by value.



e.g.: /* A simple C program containing a function that alters the

value of its argument. */

```
#include <stdio.h>
```

```
main()
```



```

{
int a = 2;
printf("\na = %d (from main, before calling the function)",a);
modify(a);
printf("\na = %d (from main, after calling the function)",a);
}

modify (int a)
{
a * = 3;
printf("\na = %d (from the function, after being modified)",a);
return;
}

output: a = 2 (from main, before calling the function)
a = 6 (from the function, after being modified)
a = 2 (from main, after calling the function)

```

The original value of a (i.e.=2) is displayed when main is executed. This value is then passed to the function modify, where it is multiplied by three and the new value of the formal argument that is displayed within the function. Finally, the value of a within main (i.e., the actual argument) is again displayed, after control is transferred back to function main from function modify.

These results show that a is not altered within main, even though the corresponding value of a is changed within modify.

Passing an argument by value has certain advantages and disadvantages.

On the positive side, it allows a single valued actual argument to be written as an expression rather than being restricted to a single variable. Moreover, if the actual argument is expressed as a single variable, it protects the value of this variable from alterations within the function.

On the negative side, it prevents information from being transferred back to the calling portion of the program via arguments. Thus, passing by value is restricted to a one-way transfer of

Information.

Call by Reference

Call by reference means sending the addresses of the arguments to the called function. In this method the addresses of actual arguments in the calling function are copied into formal arguments of the called functions. Thus using these addresses we would have an access to the actual arguments and hence we would be able to manipulate them. Using a call by reference intelligently, it is possible to make a function return more than one value at a time, which involves the study of pointer.

7.12 Function Prototype

Before defining the function, it is desired to declare the function along with its prototype. In function prototype, the return value of function, type, and number of arguments are specified.

The declaration of all functions statement should be first statement in main().

The general form of function declaration using ANSI Prototype is

```
data_type function_name (type1 arg1, type2 arg2 - - - );
```

where arg1, arg2. . . are the list of arguments.

Function prototypes are desirable because they facilitate error checking between calls to a function and corresponding function definition. They also help the compiler to perform automatic type conversions on function parameters. When a function is called, actual arguments are automatically converted to the types in function definition using normal rules of assignment.

7.13 Recursive Functions

Recursion is a process by which a function calls itself repeatedly, until some specified condition has been satisfied. The process is used for repetitive computation in which each action is stated in terms of previous result.

In order to solve a problem recursively, two conditions must be satisfied:

1. The problem must be written in recursive form.
2. The problem statement must include a stopping condition.

Example: /*To calculate the factorial of an integer recursively */

```
# include <stdio.h>

main( )
{
    int n;
    longint fact (int);
    printf ("\n n = ");
    scanf ("%d", &n);
    printf ("\n n! = %ld" fact (n));
}

longint fact (int n)
{
    if (n <= 1)
        return 1;
    else
        return (n * factorial (n-1));
}
```

7.14 Library functions

The C programming language comes with a set of standard library functions that perform a variety of useful tasks. Library functions implement all input and output operations (e.g., writing to the terminal) as well as all math operations (e.g., sine and cosine evaluation).

It is important to call the proper header file at the start of the programme in order to use a library function. All of the functions in the library in question have a header file that tells the programme their name, type, and number and type of arguments. The preprocessor statement calls a header file. #include<filename>

where filename represents the name of the header file.

A library function is accessed by simply writing the function name, followed by a list of arguments, which represent the information being passed to the function. The arguments must be enclosed in parentheses, and separated by commas: they can be constants, variables, or more complex expressions. Note that the parentheses must be present even when there are no arguments.

Library Functions in Different Header Files

C Header Files

<assert.h>	Program assertion functions
<ctype.h>	Character type functions
<locale.h>	Localization functions
<math.h>	Mathematics functions

Programming Methodologies

<setjmp.h>	Jump functions
<signal.h>	Signal handling functions
<stdarg.h>	Variable arguments handling functions
<stdio.h>	Standard Input/Output functions
<stdlib.h>	Standard Utility functions
<string.h>	String handling functions
<time.h>	Date time functions

Summary

- In this unit, we learnt about “Functions”: definition, declaration, prototypes, types, function calls datatypes and storage classes, types function invoking and lastly Recursion.
- All these subtopics must have given you a clear idea of how to create and call functions from other functions, how to send values through arguments, and how to return values to the called function.
- We have seen that the functions, which do not return any value, must be declared as “void”, return type.
- A function can return only one value at a time, although it can have many return statements.
- A function can return any of the data type specified in ‘C’.

Keywords

Call by Reference: It means sending the addresses of the arguments to the called function.

Data types: It refers to the type of information while storage class refers to the life-time of a variable and its scope within the program.

Function Call: A function can be called by specifying its name followed by a list of arguments enclosed in parentheses and separated by commas.

Return Statement: Information is returned from the function to the calling portion of the program via return statement.

Self Assessment

- Which one is not a library function
 - printf ()
 - scanf ()
 - gets ()
 - abc ()
- Function call is _____
 - Print the statement
 - Use header file in program
 - calling a function whenever it is required in a program
 - None of these
- Functions are used to ____

- A. Enhances the logical clarity of the program.
- B. Helps to avoid repeated programming across programs.
- C. Helps to avoid repeating a set of statements many times.
- D. All of above

4. Scope of variable in C is_____

- A. local
- B. global
- C. intermediate
- D. both a and b

5. Variable used inside function is called _____

- A. Global variable
- B. Local variable
- C. Both a and b
- D. None of these

6. In program a and b is a_____

```
int sum()  
{  
int a=15, b =20;  
returnx+y;  
}
```

- A. Global variable
- B. Local variable
- C. Intermediate variable
- D. None of above

7. int x and int y is a_____

```
add(intx,int y)  
{  
int z;  
    z=x+y;  
printf("result is= %d",z);  
}
```

- A. formal Parameter
- B. actual Parameter
- C. intermediate
- D. both a and b

8. Actual parameter is_____

- A. parameters that appear in function calls.
- B. parameters that appear in function definition.